

# **IA Aplicada com LLMs**

## **Aula 04 — O agente e o estado**

## Onde paramos

O deck anterior passou meia hora dizendo que vocês deveriam escrever um **workflow**.

Este é sobre o caso em que não devem.

## Quando o agente é a resposta certa

Três condições, **todas** ao mesmo tempo:

1. A tarefa é **aberta** — objetivo claro, caminho não
2. O **número de passos** é desconhecido
3. O ambiente devolve **feedback verificável**

## A terceira quase nunca é dita — e é a que decide

Um agente funciona porque **erra e se corrige**.

Sem sinal de erro vindo do mundo, o laço não corrige nada:  
ele acumula passos sobre uma premissa errada, com a confiança intacta.

| Tarefa                                       | Feedback?                 |
|----------------------------------------------|---------------------------|
| Corrigir bug — o teste passa ou falha        | <b>sim</b> , binário      |
| Investigar despesa — os valores batem ou não | <b>sim</b> , parcial      |
| Escrever texto persuasivo                    | <b>não</b> → não é agente |

# Parte 1

**O problema**

## O laço da Aula 03 guarda tudo aqui

```
mensagens = [  
    {"role": "system", "content": SYSTEM},  
    {"role": "user", "content": pergunta},  
]
```

Estamos na mensagem 40. Respondam, olhando **só para a lista**:

## Sete perguntas que a lista não responde

| Pergunta                       | Onde está?                                            |
|--------------------------------|-------------------------------------------------------|
| Quantos passos já foram dados? | dá para <i>inferir</i> , contando                     |
| Quantos tokens foram gastos?   | <b>não está</b> — o <code>usage</code> foi descartado |
| Quanto custou?                 | idem                                                  |
| Qual era o objetivo?           | soterrado na mensagem 2                               |
| A ferramenta X já falhou?      | como <b>texto</b> , dentro de <code>content</code>    |
| Alguma chamada se repetiu?     | varrendo e reparseando JSON                           |
| <b>Por que o agente parou?</b> | <b>em lugar nenhum</b>                                |

## O diagnóstico

**mensagens[]** é o estado E é o transporte

Ninguém guarda o estado de um pedido dentro do JSON que vai para o cliente HTTP.

Mas é exatamente isso que o laço da Aula 03 faz.



# Parte 2

## A virada

# O objeto de estado

```
@dataclass
class Estado:
    objetivo: str # imutável, a âncora
    passos: list[Passo] = field(default_factory=list)
    tokens_gastos: int = 0
    custo_estimado: float = 0.0
    ferramentas_ativas: list[str] = field(default_factory=list)
    termino: Termino | None = None
    resposta: str | None = None
    historico: list[dict] = field(default_factory=list) # DERIVADO
```

É a definição de agente da Aula 01 (§1.2) virando `dataclass`.

## Três decisões que não são óbvias

**objetivo** é campo, não a primeira mensagem

→ sobrevive a qualquer compaction

**passos** guarda ferramenta e argumentos em campos, não em texto

→ detectar laço vira comparação de tuplas, não *parsing*

**termino** é obrigatório

→ um agente que parou sem dizer por quê é indepurável

## A inversão

```
def montar_mensagens(estado: Estado) -> list[dict]:  
    """A lista de mensagens é uma VISÃO do estado."""  
    msgs = [{"role": "system", "content": SYSTEM},  
            {"role": "user", "content": estado.objetivo}]  
    msgs.extend(estado.historico)  
    if estado.n_passos % REANCORAR_A_CADA == 0:  
        msgs.append({"role": "user",  
                    "content": f"Lembrete do objetivo: {estado.objetivo}"})  
    return msgs
```

O histórico deixou de ser a verdade. Virou **formato de transporte**.

## O laço, agora

```
def rodar(objetivo: str, orcamento: Orcamento) -> Estado:
    estado = Estado(objetivo=objetivo, ferramentas_ativas=list(FERRAMENTAS))
    while True:
        if (motivo := orcamento.excedido(estado)):
            estado.termino = Termino.ORCAMENTO
            return estado
    ...
```

1. Devolve **Estado**, não **str**
2. **while True** com **saídas nomeadas**, no lugar de **for range()**
3. **historico** é campo → pode ser **reescrito**

**Isso não deixa o agente mais inteligente**

**Deixa o agente observável**

...que é a pré-condição de todo o resto.

## O que o estado destrava

| Salvaguarda              | Precisa de                              |
|--------------------------|-----------------------------------------|
| Orçamento em 4 moedas    | tokens, custo, passos, tempo            |
| Motivo de término        | <code>termino</code>                    |
| Erro recuperável × fatal | <code>Passo.erro</code>                 |
| Detecção de laço         | ferramenta + argumentos separados       |
| Reancoragem              | <code>objetivo</code> fora do histórico |
| Checkpoint               | estado serializável                     |
| Compaction               | <code>historico</code> isolado          |

Sete salvaguardas, **um** pré-requisito.

## **Parte 3**

### **Duas consequências**



## Ferramentas por fase

```
FASES = {  
    "coleta": ["buscar_despesa", "consultar_politica"],  
    "analise": ["consultar_politica", "consultar_historico"],  
    "registro": ["registrar_parecer"], # escrita, e SÓ ela  
}
```

Na fase `coleta`, `registrar_parecer` **não existe** para o modelo.

Instrução ele pode ignorar. Ferramenta não declarada, não.

## Restrição por arquitetura > restrição por prompt

E o ganho de custo vem junto: cada declaração viaja em **todas** as voltas.

20 ferramentas × ~120 tokens = ~2.400 tokens por volta.

## Humano no laço — a versão errada

```
if chamada.function.name == "registrar_parecer":  
    if input("confirma? [s/N] ") != "s":      # não façam isto  
        ...
```

Prende um processo esperando gente. Não sobrevive a reinício.

Não existe quando o agente roda atrás de uma fila.

## Humano no laço — a versão certa

```
if nome in EXIGE_CONFIRMACAO:  
    estado.termino = Termino.HUMANO  
    estado.pendencia = {"ferramenta": nome, "argumentos": argumentos}  
    salvar_checkpoint(estado)  
    raise PausaParaHumano(estado)
```

Confirmação é **uma das quatro formas de terminar**.

O agente para, grava e devolve o controle. Retoma horas depois.

## E tem preço

Cada ponto de confirmação **mata um pedaço da autonomia** que justificava o agente.

Um agente que pede confirmação a cada passo é um assistente de digitação caro.

Confirmação vai onde o erro é irreversível. Em nenhum outro lugar.

# Síntese

- Agente exige três condições — inclusive **feedback verificável**
- `mensagens[]` é transporte, **não é estado**
- O objeto de estado é a definição da Aula 01 virando `dataclass`
- O histórico passa a ser **derivado** — e por isso reescrevível
- Sete salvaguardas dependem dessa separação
- Exponha só as ferramentas da fase: **arquitetura vence prompt**
- Confirmação humana é terminação com checkpoint, não `input()`